

Relatório 17 - Redes Neurais Convolucionais 2 (Deep Learning) (II)

Felipe Fonseca

1. Introdução

Nesse card foi estudado novamente os conceitos de redes neurais convolucionais, tendo exemplos de uso com imagens e textos, utilizando de conceitos como camadas de convolução, pooling, embedding, apresentando boas práticas de uso e contendo exemplos práticos de códigos.

2. Desenvolvimento

Novamente nesse Card é apresentado os conceitos de convolução e redes neurais convolucionais. Convolução é a tática de pegar um objeto e aplicar um filtro sobre ele com o objetivo de obter suas principais características. Utilizando de exemplo uma imagem, utilizamos filtros que colocam blur, ou então detectam as bordas da imagem.

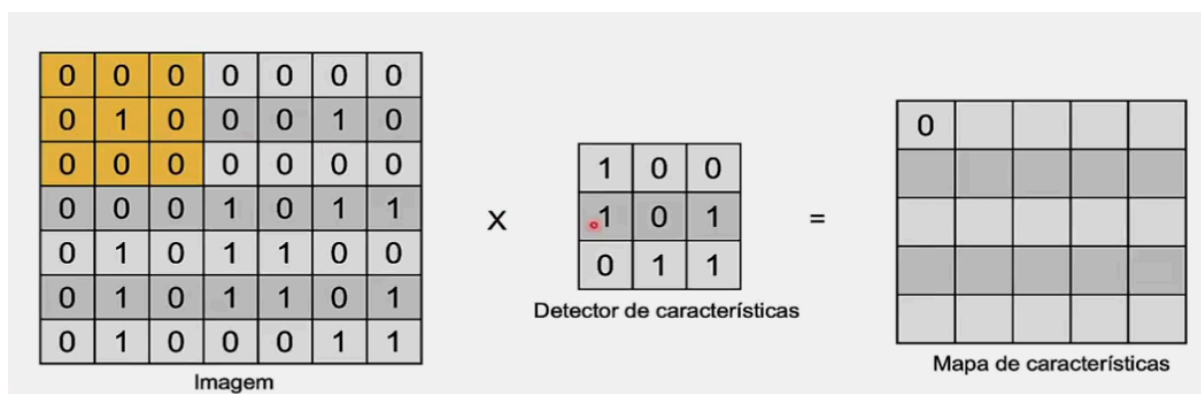


Imagem 2 (retirada da videoaula)

A convolução é feita da forma que é mostrada na imagem acima. O filtro é aplicado na matriz e desliza sobre ela, a cada parte, ela faz a multiplicação de cada valor de forma sequencial, fazendo depois a soma dos valores, e aplicando no resultado. Porém existem algumas formas diferentes de fazer isso.

A forma tradicional, também chamada de "valid", é onde aplicamos o detector de características sobre a imagem inteira, é exatamente o tipo que está sendo exibido na imagem acima.

A segunda forma é denominada "same", onde o objetivo é que a matriz de saída tenha o mesmo tamanho da matriz de entrada. Isso pode ser útil em casos onde é

necessário múltiplas convoluções, pois com o método tradicional, isso faria a matriz sumir. Para a imagem manter o mesmo tamanho de saída, é feito de forma imaginária que existe um preenchimento de Zeros envolta da matriz, causando no final que a matriz de saída mantenha o mesmo tamanho. Exemplo na imagem abaixo:

0	0	0	0	0	0	0	0
0	3	3	4	4	7	0	0
0	9	7	6	5	8	2	0
0	6	5	5	6	9	2	0
0	7	1	3	2	7	8	0
0	0	3	7	1	8	3	0
0	4	0	4	3	2	2	0
0	0	0	0	0	0	0	0

$6 \times 6 \rightarrow 8 \times 8$

*

1	0	-1
1	0	-1
1	0	-1

=

3×3

-10	-13	1			
-9	3	0			

6×6

Imagem 2

A terceira forma é chamada "full", nesta forma, o filtro desliza sob a matriz inicial imaginando que existam mais 2 preenchimentos com 0, acarretando no resultado final, que terá um tamanho maior que a matriz inicial. Esse método quase nunca é utilizado, exceto quando o caminho é inverso, por exemplo, ao criar uma rede neural que gere imagens. Exemplo na imagem abaixo:

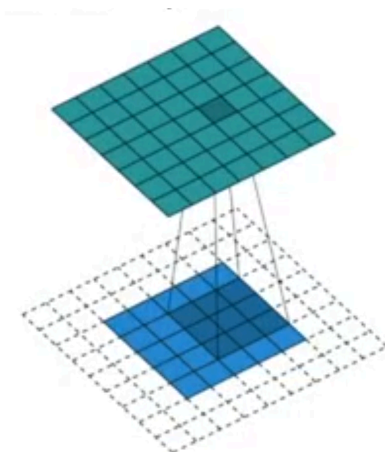
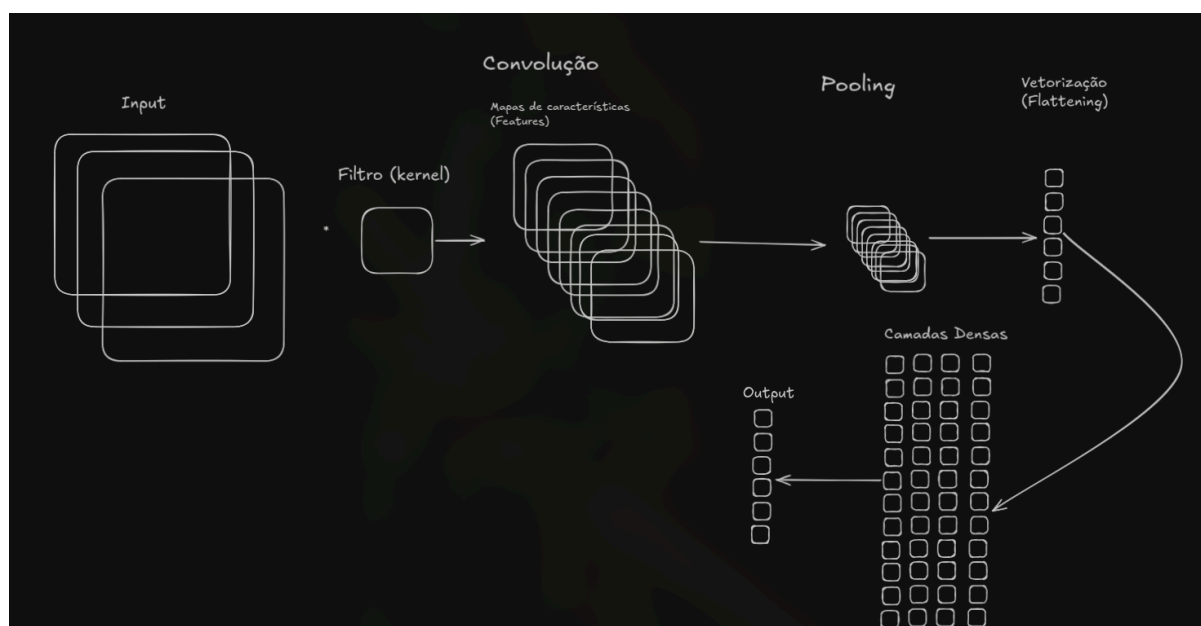


Imagem 3 (retirada da videoaula)

Uma tática que é citada pelo instrutor na videoaula, é o compartilhamento de pesos na rede neural, isto é, o conjunto de pesos do filtro que é aplicado na matriz é fixo para o toda a matriz. Isto funciona porque um objeto continua sendo o mesmo objeto independente em qual canto da imagem ele está, então é juntar o útil ao agradável.

É também citado o funcionamento para imagens 3D, onde o filtro desliza sob a matriz 3D, obtendo como resultado um mapa de características 2D. Para resolver o problema, são utilizados vários filtros, para observar diferentes características.

A arquitetura de uma CNN pode ser dividida em 2 partes, onde a primeira parte é a abstração do objeto, podendo ser frases, imagens, áudio, etc, e a segunda parte é composta pelas camadas densas, que funcionam para classificar os objetos.



Algumas recomendações quanto a arquitetura:

- Utilizar um aumento dos canais de convolução, isto é por exemplo, utilizar inicialmente 32 filtros na primeira camada de convolução, depois 64 na segunda camada, depois 128 na terceira.
- Utilizar blocos, por exemplo: convolução, depois pooling, convolução, pooling, ou então, convolução, convolução, pooling.
- Utilizar stride invés de pooling, isto é, na hora de fazer a convolução, fazer um salto de pixels para que a imagem final tenha um resultado ainda menor, é uma redução da imagem de forma eficiente.

Também foi mostrado o Data Augmentation, que é uma solução para criar mais imagens para treinar o modelo. Ela é útil quando não há imagens o suficientes, mas também é útil para criar imagens com padrões diferentes, como por exemplo, deixando um carro de cabeça para baixo, mas que ainda sim, a IA precisa reconhecer que é um carro.

E por fim, o Batch Normalization pode ser uma camada utilizada entre as camadas de convolução. Normalmente normalizamos os dados antes de iniciarmos as redes neurais, mas dentro das redes, os dados podem fugir da normalização, por isso, colocar camadas de normatizações também pode ajudar a evitar o overfitting.

O próximo passo foi o entendimento de Embedding. Embedding é a solução que surgiu para categorizar textos, sendo uma alternativa ao One Hot Encoder. Enquanto o One Hot Encoder cria um vetor gigante de zeros ou uns para um único atributo, na qual o tamanho do vetor seria nesse caso, o tamanho de palavras do dicionário, o Embedding utiliza um vetor fixo, na qual ele mantém palavras com significados similares com valores similares no mapa vetorial. Ou seja, a palavra Rei terá um valor próximo ao valor Rainha, mas terá um valor mais longe para a palavra Carro. A convolução de um Embedding funciona de forma semelhante a de uma imagem, a diferença é que o filtro tem apenas uma linha.

Depois é apresentado bastante sobre os cálculos de erros que são mais utilizados em uma rede neural.

O primeiro foi o MSE, que é o Minimum Squared Error, ou Erro Quadrático Médio. Dentre os motivos que ele é utilizado está que ao elevar o número ao quadrado, o erro fica sempre positivo, independente se o erro foi pra mais ou menos. Porém ele conta com algumas características. Por exemplo, ele assume que os erros estão conforme uma distribuição normal de probabilidade, ou sejam caso os erros estejam mais de acordo com outra função de probabilidade, talvez seja necessário utilizar outro método para calcular o erro.

O segundo apresentado foi a Binary Cross Entropy, que utiliza a Distribuição de Bernoulli, que tem justamente como característica ter uma resposta binária.

O terceiro é o Categorical Cross-Entropy, que utiliza a distribuição categorial de probabilidade, que invés de voltar um valor binário 1 e 0, ela retorna K resultados com valores de probabilidade entre 0 e 1, considerando que a soma de todos os K valores da exatamente 1.

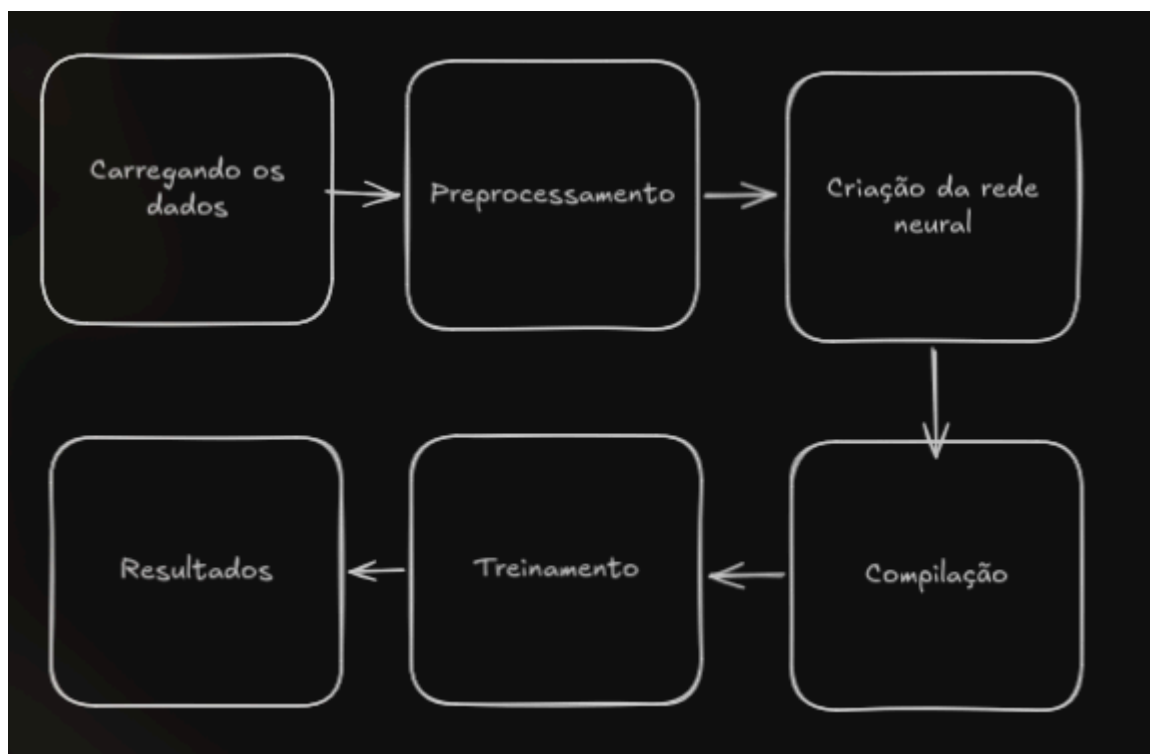
Depois foi explicado os conceitos importantes sobre a descida do gradiente. De forma resumida, a descida do gradiente será o algoritmo que vai ajustar os pesos para diminuir o erro até chegar no erro mínimo. O SGD é o método de minimizar o peso do computador ao fazer o processo da descida do gradiente, invés de utilizar todos os dados para calcular o erro, ele faz uma estimativa com pequenos grupos, permitindo que os pesos sejam atualizados várias vezes na mesma época.

E por último, foi falado um pouco do caminho dos otimizadores de uma rede neural. O principal problema era encontrar o passo ideal, pois no início são necessários passos largos, porém ao final é preciso de passos curtos. A evolução passou por algumas técnicas, como o Momentum, que é a técnica de adicionar mais uma variável de velocidade no cálculo de erro, ou seja, a atualização não é mais dependente apenas do gradiente atual, ela leva os outros cálculos anteriores em conta, de forma resumida, ela acumula a direção. O Adagrad contém na fórmula a divisão da taxa de aprendizado, o que aumenta o passo, mas faz que a taxa de

aprendizado vire 0 muito rapidamente. o RMSprop utiliza uma média móvel, que resolve o problema do Adagrad, mas ainda não é perfeito.

O Adam foi o otimizador que surgiu para juntar o RMSprop e o Momentum, sendo no momento o padrão da indústria e o otimizador mais utilizado.

Para a prática, eu utilizei o dataset Reuters, que é um dos datasets já disponíveis no tensor flow. Ele nada mais é do que um dataset que contém notícias da agência Reuters, que são classificadas em 46 tópicos diferentes.



Carregando os dados: fazendo a importação dos dados do dataset, dados de treino e teste.

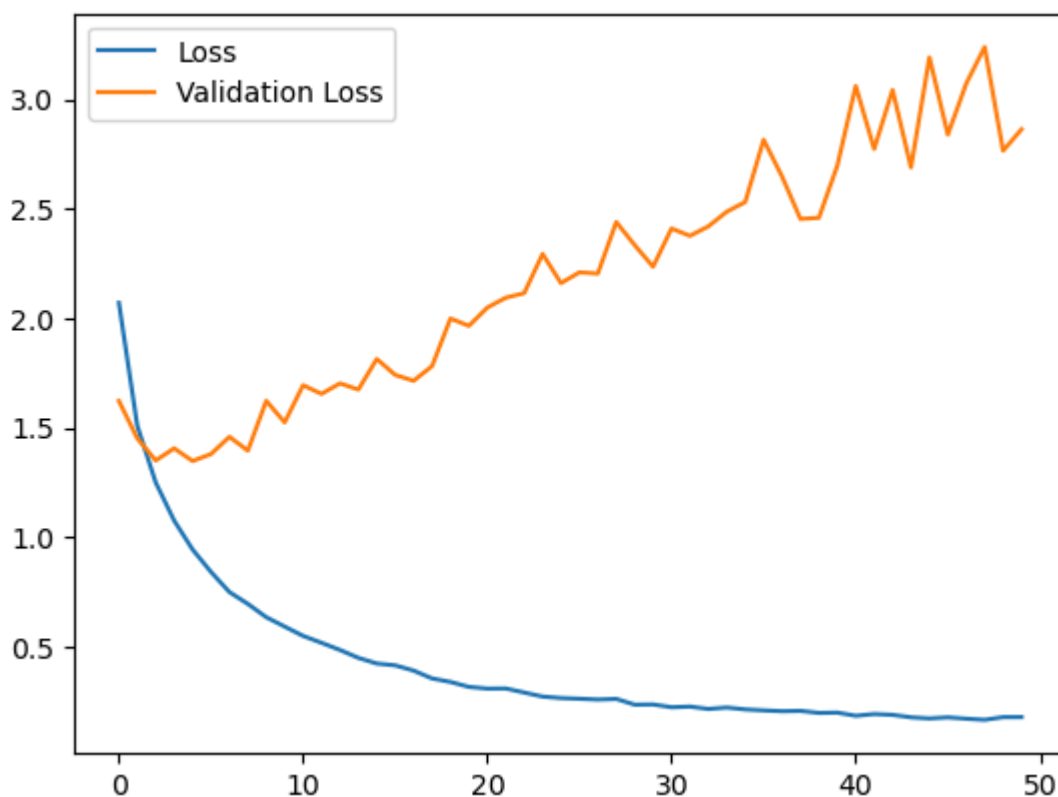
Pré Processamento: padding das sequências de palavras e transformação das labels em categórico.

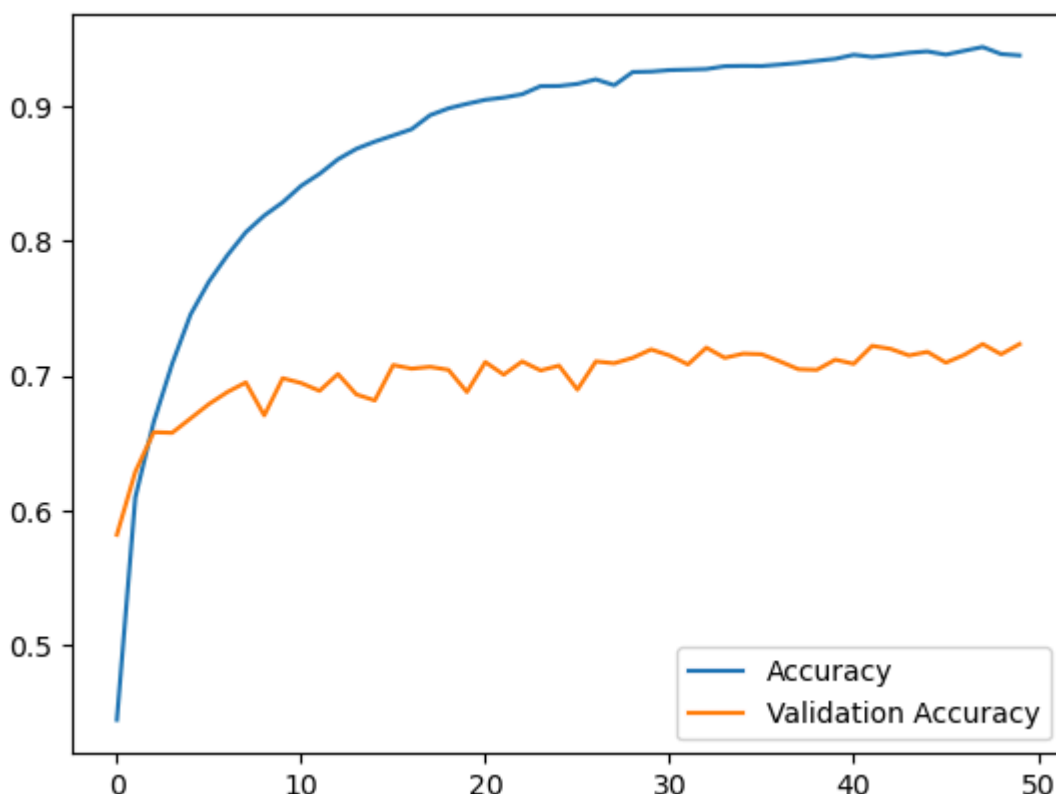
Criação da rede neural: camada de Embedding, seguida de camadas de convolução e Pooling intercaladas.

Compilação: utilizei categorical crossentropy pois era o mais indicado para um resultado categórico.

Treinamento: realizado o treinamento dos dados juntamente com a validação.

Resultado; os resultados não foram bons. Apesar de um accuracy muito alto, e de uma loss baixa, os valores da validação foram o contrário, como é possível ver nos gráfico conforme o treinamento:





Esses gráficos são o comportamento clássico de overfitting, ou seja, o modelo não aprendeu de verdade, apenas se adaptou aos dados.

3. Conclusão

Nesse card foi estudado mais sobre as redes neurais convolucionais, apresentando alguns outros conceitos a mais do que já haviam citados anteriormente. A parte principal a ser acrescentada foi o embedding, que mostra outra visão de como lidar com dados como os textos. Além disso, foi bem apresentado os conceitos e funcionamentos de cálculos de perda e otimização que anteriormente só haviam sido citados sem explicação exata do que faziam.

4. Referências

Imagem 1 retirada da videoaula.

Imagem 2 retirada de: <<https://datahacker.rs/what-is-padding-cnn/>>

Imagem 3 retirada da videoaula.

Dataset encontrado no site oficial do keras: <<https://keras.io/api/datasets/reuters/>>

Informações e dicas do dataset: <<https://martin-thoma.com/nlp-reuters/>>

Videos de aula do Card.